

Offline-First QR-Based Traceability and Predictive Analytics for Sri Lankan Kithul Supply Chains

Component Focus: Network Suitability Prediction for Adaptive Offline-First Synchronization

Group ID: R26-IT-078

Project Proposal Report

SAMARATHUNGA S.H.P.N.D - IT22576484

B.Sc. (Hons) Degree in Information Technology Specialized in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology, Sri Lanka

March 2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
SAMARATHUNGA S.H.P.N.D	IT22576484	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Name	Date	Signature

ABSTRACT

Rural kithul supply chains in Sri Lanka require mobile data collection from farmer locations where field collectors frequently operate under unstable cellular connectivity. Offline-first storage enables field records to be captured without a signal, but it does not fully solve the synchronization problem because conventional mobile applications often attempt to upload pending data as soon as any connection is detected. In weak rural coverage zones, this can cause repeated upload failures, delayed visibility of critical can and quality records, wasted mobile data, and unnecessary battery consumption. This proposal therefore focuses on Network Suitability Prediction for Adaptive Offline-First Synchronization. The component extends a Flutter-based offline-first mobile synchronization backbone by integrating a machine learning layer trained on real public mobile network performance data, including Ookla mobile performance tiles and supporting cell-tower infrastructure data from OpenCelliD. The proposed model predicts area-wise synchronization suitability using features such as upload speed, download speed, latency, tile reliability indicators, temporal patterns, and tower-density features. The prediction output is used by an adaptive synchronization scheduler to decide whether the mobile app should synchronize all pending records, synchronize only high-priority records, reduce JSON delta-bundle size, or keep records offline until a better network zone is reached. Local SQLite storage supports offline event capture, outbox management, checkpoint-based recovery, and priority tagging, while the AWS backend validates idempotent bundles and updates server-side projections for connected desktop and analytics modules. Expected outcomes include a real-data-driven network suitability map, a trained suitability prediction model, an adaptive sync strategy recommendation mechanism, reduced failed synchronization attempts, improved timely upload of critical field records, and continued audit-friendly convergence between delayed mobile events and cloud projections.

Keywords: offline-first mobile systems, network suitability prediction, adaptive synchronization, mobile network performance, machine learning, kithul supply chain.

ACKNOWLEDGEMENT

I gratefully acknowledge the guidance of my supervisor and co-supervisor, whose academic advice, technical critique, and encouragement are essential to the development of this research proposal. I also thank the industry stakeholders and domain participants associated with the Sri Lankan kithul supply chain for providing practical insight into rural field collection, connectivity limitations, and traceability needs. My sincere appreciation is extended to my group members for constructive discussions on system integration, and to my family and friends for their patience and support throughout the preparation of this proposal.

TABLE OF CONTENT

DECLARATION

ABSTRACT

ACKNOWLEDGEMENT

TABLE OF CONTENT (Auto Generated)

LIST OF TABLES

LIST OF FIGURES

LIST OF ABBREVIATIONS

LIST OF APPENDICES

1. INTRODUCTION

1.1 Background and Context

1.2 Literature Review

1.3 Research Gap

2. OBJECTIVE

2.1 Main Objective

2.2 Specific Objectives

3. METHODOLOGY

4. HIGH-LEVEL SYSTEM ARCHITECTURE

5. USER REQUIREMENTS

6. COMMERCIALIZATION PLAN

7. BUDGET AND JUSTIFICATION

8. WORK BREAKDOWN STRUCTURE (WBS)

9. GANTT CHART

10. REFERENCES LIST

11. APPENDICES

LIST OF TABLES

Table 1-1: Comparative review of offline-first, synchronization, network data, and ML approaches

Table 3-1: Candidate features and targets for network suitability prediction

Table 3-2: Evaluation metrics for the proposed network suitability and synchronization component

Table 5-1: Functional requirements of the component

Table 5-2: Non-functional requirements and target values

Table 7-1: Estimated budget and justification

Table 8-1: Work packages and milestones for the component

Table 9-1: Proposed semester Gantt chart for the component

Table 11-1: Proposed network suitability dataset schema

Table 11-2: Proposed mobile-local and cloud schema summary

LIST OF FIGURES

Figure 4-1: High-level architecture of network-suitability-aware offline synchronization

Figure 4-2: Real-data-driven network suitability prediction pipeline

Figure 4-3: Adaptive synchronization decision flow using predicted suitability

Figure 4-4: Mobile sync assistant and network suitability map concept

Figure 8-1: Work Breakdown Structure for the component

Figure 9-1: Semester Gantt chart for the component

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
AWS	Amazon Web Services
CRDT	Conflict-free Replicated Data Type
CRUD	Create, Read, Update, Delete
EPCIS	Electronic Product Code Information Services
GIS	Geographic Information System
GPS	Global Positioning System
HLC	Hybrid Logical Clock
HTTP	Hypertext Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
JWT	JSON Web Token
LWW	Last Write Wins
ML	Machine Learning
PostGIS	PostgreSQL Geographic Information System Extension
QR	Quick Response
REST	Representational State Transfer
RTT	Round-Trip Time
SDG	Sustainable Development Goal
SLA	Service Level Agreement
UI	User Interface
UUID	Universally Unique Identifier

LIST OF APPENDICES

Appendix A: Proposed Network Suitability Dataset Schema

Appendix B: Example Adaptive Synchronization Decision Manifest

Appendix C: Eraser DiagramGPT Prompts for Required Diagrams

Appendix D: Proposed Mobile-Local and Cloud Schema Summary

1. INTRODUCTION

1.1 Background and Context

Sri Lankan kithul supply chains remain operationally fragmented. In many production areas, sap collection, transport, collection-center intake, processing into treacle or jaggery, and buyer-order fulfilment are still coordinated through paper logbooks, handwritten tickets, phone calls, and disconnected spreadsheets. The proposed group project introduces a digital platform in which QR-based can identification, mobile field collection, factory quality checks, spoilage prediction, and demand forecasting depend on timely and trustworthy synchronization of field records.

The field collector is the stakeholder most affected by connectivity instability. A collector may visit several rural farmer locations, scan or select can records, enter quantity and field-level quality data, and then travel toward the collection center or factory. The mobile app must work without an active Internet connection, but the business value is only realized when pending records eventually reach the cloud backend and become visible to factory staff and downstream analytics modules.

Offline-first storage alone is therefore insufficient. A conventional offline-first app normally stores records locally and tries to synchronize when a connection is detected. However, a weak mobile connection may technically exist while still being unsuitable for a reliable upload. In rural field conditions, attempting synchronization in such areas can repeatedly fail, increase retry counts, waste mobile data and battery, delay high-priority records, and create unnecessary recovery overhead for the backend.

Public connectivity statistics confirm the relevance of this problem. The Department of Census and Statistics reports sector differences in digital literacy in Sri Lanka, while the International Telecommunication Union identifies persistent gaps between urban and rural Internet access and mobile network coverage [15], [16]. In addition, the existence of national mobile broadband subscriptions does not guarantee stable upload performance at every farmer location. For field synchronization, upload speed and latency are more important than merely detecting a network interface.

This proposal therefore changes the component focus from a purely technical offline synchronization mechanism into Network Suitability Prediction for Adaptive Offline-First Synchronization. The component uses real-world mobile network performance data, mainly Ookla mobile performance tiles, and optional OpenCellID cell tower data, to predict whether the current or nearby location is suitable for synchronization [11], [13]. The prediction is then used by the mobile sync controller to choose a strategy: sync all pending records, sync only high-priority records, reduce bundle size, or keep records safely offline until the collector reaches a better network zone.

The component aligns with the Software Systems & Technologies research cluster because it addresses mobile-edge synchronization, geospatial network intelligence, machine learning-based system adaptation, distributed event storage, and resilient software architecture under real deployment constraints. It also supports SDG 2, SDG 8, SDG 9, and SDG 12 by improving data reliability, reducing operational waste, and strengthening digital infrastructure for smallholder-oriented agro-processing supply chains.

1.2 Literature Review

Digital traceability is a core requirement for modern food and agricultural supply chains. ISO 22005 and GS1 EPCIS provide structured principles and event models for traceability, while food traceability research emphasizes transparency, recall readiness, and quality assurance [1]-[3]. These standards and studies clarify what should be traced, but they do not prescribe how a mobile field application should choose the best synchronization strategy under unstable rural cellular conditions.

Offline-first and local-first systems address the need for software to remain usable without a continuous cloud connection. Kleppmann et al. argue that local-first software should support offline usability, local data ownership, and later synchronization [4]. This principle fits kithul field operations because data capture should not stop when network coverage is lost. However, local-first principles do not by themselves determine whether a weak network is suitable for uploading pending records or how important records should be prioritized.

Synchronization and replicated data research provides methods for efficient convergence. Differential synchronization, delta-state CRDTs, and optimistic replication demonstrate that systems can reduce payload size and tolerate delayed or reordered updates [5]-[9]. These studies support the use of delta bundles, checkpoints, idempotency, and correction-aware event histories. However, they mostly focus on generic synchronization or collaborative data structures rather than on using real geospatial network performance data to drive application-level sync decisions.

Commercial offline-capable platforms also provide useful lessons. Cloud Firestore supports offline persistence and automatically synchronizes local changes when a device comes back online, but multiple changes to the same document are resolved using last-write-wins semantics [19]. CouchDB replication supports incremental database synchronization and checkpointed recovery, but its document revision model does not provide a kithul-specific sync-suitability decision layer [20]. AWS AppSync supports configurable conflict detection and resolution strategies such as optimistic concurrency, automerger, and Lambda handlers [21]. These products simplify offline development, but they do not target a rural agricultural workflow where the mobile app must predict where and when synchronization is most likely to succeed.

Real public mobile network datasets make it possible to introduce this missing intelligence. Ookla Open Data provides global fixed and mobile network performance tiles with average download speed, upload speed, latency, test count, device count, and tile geometry; the mobile layer represents tests from cellular connections with GPS-quality location accuracy [11]. The AWS Open Data Registry also describes the same dataset as global fixed broadband and mobile network performance allocated to approximately 610.8 m web-mercator tiles and updated quarterly [12]. OpenCellID provides an open database of cell tower positions and metadata that can be used to derive tower-density and nearest-tower-distance features [13].

Machine learning has already been applied to mobile network performance prediction. For example, Eyceyurt et al. used LTE and 5G radio parameters to estimate uplink throughput using learning algorithms, and other IEEE-published work has studied throughput prediction in LTE and 5G networks [17], [18]. These studies show that throughput and latency prediction are valid machine learning problems. The novelty in this proposal is not claiming that network performance prediction

is entirely new; rather, the contribution is applying real-data-driven network suitability prediction to control synchronization behavior inside an offline-first rural kithul field collection application.

Table 1-1: Comparative review of offline-first, synchronization, network data, and ML approaches

Approach / Product	Representative Source	Main Strength	Limitation in Rural Kithul Context
Standards-based traceability	[1]-[3]	Provides event vocabulary, source visibility, and supply-chain traceability principles.	Does not define mobile-edge synchronization strategy under weak rural connectivity.
Local-first / offline-first software	[4]	Supports uninterrupted user work during disconnection.	Does not decide when or where delayed mobile data should be synchronized.
Differential / delta synchronization	[5], [7]	Reduces payload size and supports incremental transfer.	Typically relies on current link behavior rather than geospatial suitability prediction.
CRDTs and optimistic replication	[6]-[10]	Enable convergence and conflict tolerance under concurrent or delayed updates.	Generic semantics can be heavy or unsuitable for bounded agricultural business events.
Managed offline-capable products	[19]-[21]	Offer fast implementation, caching, replication, and conflict strategies.	Do not provide a kithul-specific priority sync scheduler driven by network suitability prediction.
Public mobile performance datasets	[11], [12]	Provide real upload speed, download speed, latency, tile geometry, test count, and device count.	Datasets must be converted into application-level suitability classes and sync decisions.
Cell tower open data	[13]	Supports infrastructure-aware features such as tower density and distance to nearest tower.	Tower presence alone does not guarantee upload suitability and must be combined with performance data.
ML-based throughput prediction	[17], [18]	Shows that mobile throughput/latency prediction can be modeled using learning algorithms.	Prior work is network-performance oriented, not integrated with rural offline sync strategy.

1.3 Research Gap

The reviewed literature reveals three related but separate research streams. First, traceability standards and agri-food traceability studies define the need for structured event capture, but they do not address unreliable mobile synchronization in rural field operations. Second, offline-first synchronization and CRDT-based replication research provides mechanisms for delayed convergence, conflict tolerance, and payload reduction, but these mechanisms generally react to current link conditions rather than using real geospatial network performance data to predict suitability before attempting uploads. Third, mobile network performance datasets and throughput prediction studies focus on connectivity analysis, but they are rarely integrated into agricultural traceability applications as a direct control input for synchronization scheduling.

Accordingly, the specific research gap is the absence of a real-data-driven network suitability layer that connects public mobile network performance data, machine learning prediction, offline-first outbox management, and rural kithul field synchronization strategy. Existing offline apps can store data locally; existing coverage maps can show weak areas; and existing ML studies can predict network throughput. What is missing is a deployment-oriented mechanism that converts predicted area-level upload suitability into concrete sync actions for field collectors, such as sync all, sync high-priority records only, reduce chunk size, or delay synchronization.

This proposal addresses that gap through a component titled Network Suitability Prediction for Adaptive Offline-First Synchronization. The contribution is a machine learning-assisted synchronization layer that uses real mobile network performance data to estimate synchronization suitability and uses the result to control an adaptive offline-first synchronization mechanism for rural kithul supply chains. The component remains compatible with event-based traceability, idempotent JSON delta bundles, checkpoints, and correction-preserving audit history, but its main research novelty is the prediction-driven sync decision layer.

2. OBJECTIVE

2.1 Main Objective

To design, implement, and evaluate, by the end of the project period, a real-data-driven network suitability prediction component for adaptive offline-first synchronization in a rural kithul field collection mobile application, using public mobile network performance data and cell-tower infrastructure features to predict whether a field location is suitable for synchronization, and using the prediction to select an appropriate sync strategy that reduces failed uploads, prioritizes critical records, and preserves reliable convergence with the cloud backend.

2.2 Specific Objectives

- To define a mobile offline-first event, outbox, checkpoint, and priority model that supports local field data capture and deferred synchronization of kithul collection records.
- To collect, clean, and preprocess real public mobile network performance data from Ookla Open Data and optional OpenCellID cell-tower datasets for Sri Lankan rural field collection regions.
- To engineer geospatial and temporal features such as tile location, upload speed, download speed, latency, test count, device count, previous-quarter performance, tower density, and distance to the nearest tower.
- To train and compare machine learning models for predicting network suitability or future upload/latency performance using real network data rather than synthetic synchronization logs.
- To develop an adaptive synchronization scheduler that uses predicted suitability to choose between sync-all, priority-only sync, reduced chunk-size sync, and delayed offline storage.
- To implement a network suitability map and mobile sync assistant UI that shows Good, Moderate, and Poor sync zones, pending records, and recommended synchronization action.
- To maintain audit-friendly event synchronization using idempotent JSON delta bundles, checkpoints, retry handling, and correction-preserving cloud validation.
- To evaluate the proposed component using ML metrics such as MAE, RMSE, accuracy, precision, recall, F1-score, and ROC-AUC, together with system metrics such as failed sync attempts, critical record upload delay, request count, and payload efficiency.

3. METHODOLOGY

Research approach

This study follows a Design Science Research approach because the outcome is a purposeful software artifact designed to solve a practical field synchronization problem. The artifact is not limited to implementing offline storage; it introduces a prediction-driven mechanism that uses real

network data to improve the timing and strategy of mobile synchronization in rural kithul collection workflows.

The work begins with problem identification in the kithul field collection process, followed by literature review, dataset selection, feature engineering, model development, prototype implementation, and evaluation. The prototype will be implemented around a Flutter mobile client, local SQLite persistence, an AWS cloud backend, PostgreSQL or PostgreSQL with PostGIS for storing validated events and suitability layers, and a connected desktop or dashboard interface that consumes server-side projections.

The mobile application will continue to support offline field capture. However, before attempting to synchronize pending outbox records, the application will obtain the current GPS location and available network state, map the location to the nearest or containing network performance tile, obtain a predicted suitability class or score, and apply a synchronization policy. This transforms offline sync from a reactive mechanism into a prediction-assisted adaptive mechanism.

Data sources and dataset preparation

The machine learning dataset will be based on real public network data rather than generated synchronization logs. The primary dataset is the Speedtest by Ookla Global Fixed and Mobile Network Performance Map Tiles, specifically the mobile cellular performance layer. The dataset includes average download speed, average upload speed, average latency, test count, unique device count, quadkey, centroid coordinates, and tile geometry. It is distributed as quarterly map tiles and can be accessed through public S3/HTTP sources described in the Ookla Open Data repository and AWS Open Data Registry [11], [12].

The second supporting dataset is OpenCellID. It provides country-specific cell-tower data that can be used to derive infrastructure-aware features such as nearest-tower distance, tower density within selected radius bands, and available radio technologies where present [13]. These features are not used as a replacement for measured performance; rather, they provide additional context that may improve model generalization in areas with sparse performance measurements.

The preprocessing pipeline will filter the mobile performance data to Sri Lanka and then to selected kithul collection districts or bounding boxes. Each tile will be represented by a quadkey, geometry, centroid latitude/longitude, year, quarter, upload speed, download speed, latency, test count, and device count. Time-series features will be created by joining the same quadkey across quarters, such as previous-quarter upload speed, rolling average upload speed, rolling latency, and performance trend. OpenCellID records will be spatially joined to each tile to compute tower-density features.

Two modeling formulations will be considered. In the regression formulation, the model predicts next-period upload speed and latency. In the classification formulation, real measured upload and latency values are converted into operational suitability classes: Good, Moderate, and Poor. The thresholds will be calibrated during the project based on the expected size of JSON delta bundles and acceptable latency for critical field records. This use of operational labels is derived from real network measurements and does not require synthetic training data.

Table 3-1: Candidate features and targets for network suitability prediction

Category	Feature Examples	Purpose
Network performance	avg_u_kbps, avg_d_kbps, avg_lat_ms, avg_lat_up_ms, avg_lat_down_ms	Estimate upload suitability and network responsiveness.
Reliability indicators	tests, devices	Measure confidence in each tile aggregate.
Spatial features	latitude, longitude, quadkey, district, nearest-tile distance	Support geospatial lookup and suitability mapping.
Temporal features	year, quarter, previous-quarter metrics, rolling averages, trends	Capture seasonal and temporal network performance changes.
Infrastructure features	nearest tower distance, tower count within 1 km/3 km, radio type	Represent supporting cell-network infrastructure.
Runtime sync context	pending record count, high-priority record count, payload size, current network type	Used by the app decision layer, not as the primary public-data training source.
Targets	next_upload_kbps, next_latency_ms, or suitability class	Define regression or classification output for the ML model.

Technical methods and algorithms

The first artifact is the mobile offline-first data layer. The Flutter client will maintain a local SQLite event store, materialized projections, a sync outbox, a checkpoint table, and a sync diagnostics table. Operational field records, such as collection events, field quality measurements, and correction events, will be saved locally even when the device is offline. Each pending record will also carry a priority level, because not all records have the same urgency during weak connectivity.

The second artifact is the network suitability model. Candidate models will include Linear Regression and Random Forest Regressor for numeric prediction of upload speed or latency, and Logistic Regression, Decision Tree, Random Forest Classifier, and Gradient Boosting Classifier for Good/Moderate/Poor suitability classification. The baseline model will be a simple rule-based threshold method, while the proposed method will use learned relationships between historical performance, spatial position, temporal quarter, reliability indicators, and tower features.

The third artifact is the adaptive sync scheduler. The scheduler will combine the predicted suitability class with current application context such as pending outbox size, record priority, and current network transport. If predicted suitability is Good, the app will synchronize all pending records using normal or larger delta bundles. If suitability is Moderate, it will synchronize high-priority records first and reduce bundle size. If suitability is Poor, it will keep records offline, notify the collector that records are safely stored, and retry later or recommend a nearby better sync zone on the map.

The fourth artifact is the cloud validation and convergence pipeline. The backend will validate idempotent JSON delta bundles, suppress duplicates, store accepted events in a cloud event log, update projection tables, and return acknowledgements plus checkpoint watermarks. Conflict handling will remain lightweight and correction-aware: operational events are appended, repeated submissions are deduplicated, and corrections are represented as explicit correction-events rather than destructive overwrites.

Validation and evaluation

Evaluation will be conducted at two levels. First, the machine learning model will be evaluated using held-out real network performance tiles and time-based or spatial cross-validation. Regression models will be evaluated using MAE, RMSE, and R-squared for upload speed and latency prediction.

Classification models will be evaluated using accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC where applicable.

Second, the synchronization strategy will be evaluated against two baselines: a conventional offline-first sync strategy that attempts upload whenever a network is detected, and a static rule-based adaptive strategy that uses only current observed link state. The proposed strategy will use the predicted network suitability class. The system will be compared using failed synchronization attempts, retry count, time to upload high-priority records, request count, payload volume, and successful recovery after interrupted transfers. Dummy kithul records may be used only to test application workflow and backend synchronization mechanics; the ML model itself will be trained using real public mobile network data.

Table 3-2: Evaluation metrics for the proposed network suitability and synchronization component

Metric	Definition / Measurement	Evaluation Target or Comparison
Upload speed MAE/RMSE	Error between predicted and actual tile upload speed	Lower than baseline regression model
Latency MAE/RMSE	Error between predicted and actual tile latency	Lower than baseline regression model
Suitability classification F1-score	Balanced performance over Good, Moderate, and Poor classes	Target ≥ 0.80 after calibration
Suitability map coverage	Percentage of target collection area covered by a prediction or nearest-tile fallback	High coverage for selected pilot area
Failed sync attempt rate	Failed uploads under proposed strategy compared with conventional sync	Lower than baseline
Critical record upload delay	Time until high-priority records reach backend projections	Lower than conventional sync in weak areas
Payload efficiency	Total bytes transmitted per 100 mobile events	Improved by priority sync and reduced bundles
Duplicate suppression rate	Retried bundles that do not create duplicate cloud effects	100% in controlled tests
Checkpoint recovery success	Interrupted uploads resumed from latest acknowledged chunk	$\geq 95\%$ in integration tests
Mobile UI response time	Time to show suitability status and sync recommendation	< 2 seconds after local lookup/model response

4. HIGH-LEVEL SYSTEM ARCHITECTURE

The proposed architecture is organized as a prediction-aware offline synchronization system. The Flutter mobile client remains offline-first and continues to support field capture when the network is unavailable. The AWS cloud backend remains the authoritative validation and projection layer. The new addition is a network suitability prediction layer that uses real public network data to inform mobile synchronization decisions before data transfer begins.

At the data layer, Ookla mobile performance tiles and OpenCelliD tower data are periodically downloaded, filtered to the target region, cleaned, spatially joined, and stored in a geospatial feature store. The machine learning pipeline trains a regression or classification model using historical tile performance and infrastructure features. The trained model or its generated suitability map is deployed to the backend and optionally cached on the mobile client for offline lookup.

At the mobile edge layer, the Flutter app contains a local SQLite event store, current-state projections, a sync outbox, a checkpoint store, a priority tag for each pending event, a network state reader, and a suitability lookup module. When the field collector creates records while offline, the data remains safely in SQLite. Before synchronization, the app reads current location and network context, obtains a suitability score, and selects a sync strategy.

At the cloud layer, API Gateway and Lambda endpoints receive idempotent JSON delta bundles, validate schema and business rules, suppress duplicates, write accepted events to PostgreSQL, and update server-side projections. Connected desktop users and other project modules consume the current validated projections. The suitability component therefore improves not only the mobile app, but also the timeliness and reliability of downstream QR traceability, sap spoilage prediction, and demand planning modules.

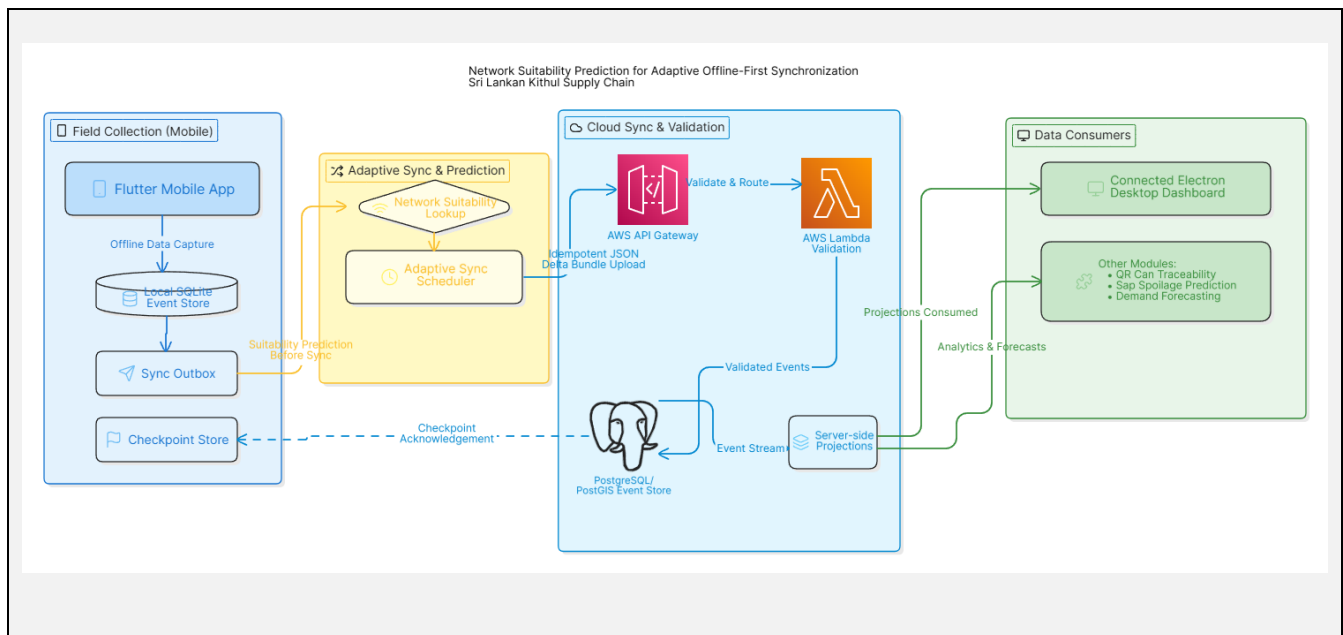


Figure 4-1: High-level architecture of network-suitability-aware offline synchronization

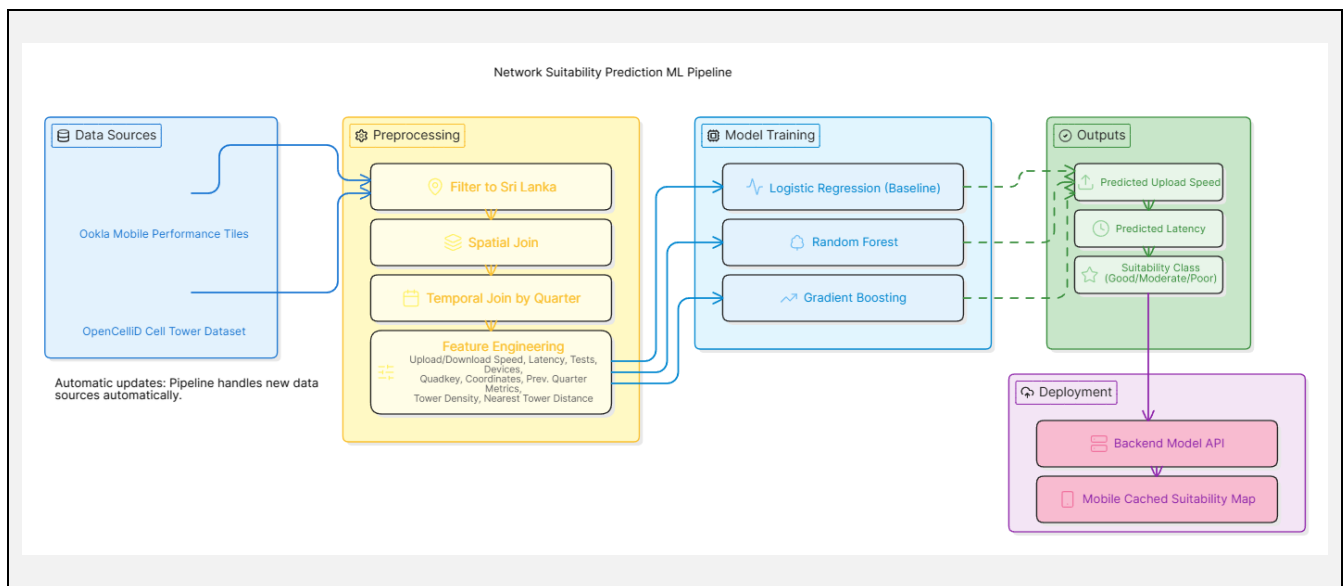
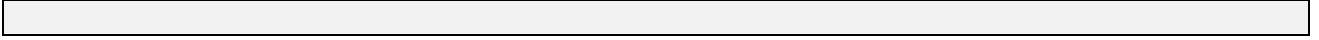


Figure 4-2: Real-data-driven network suitability prediction pipeline



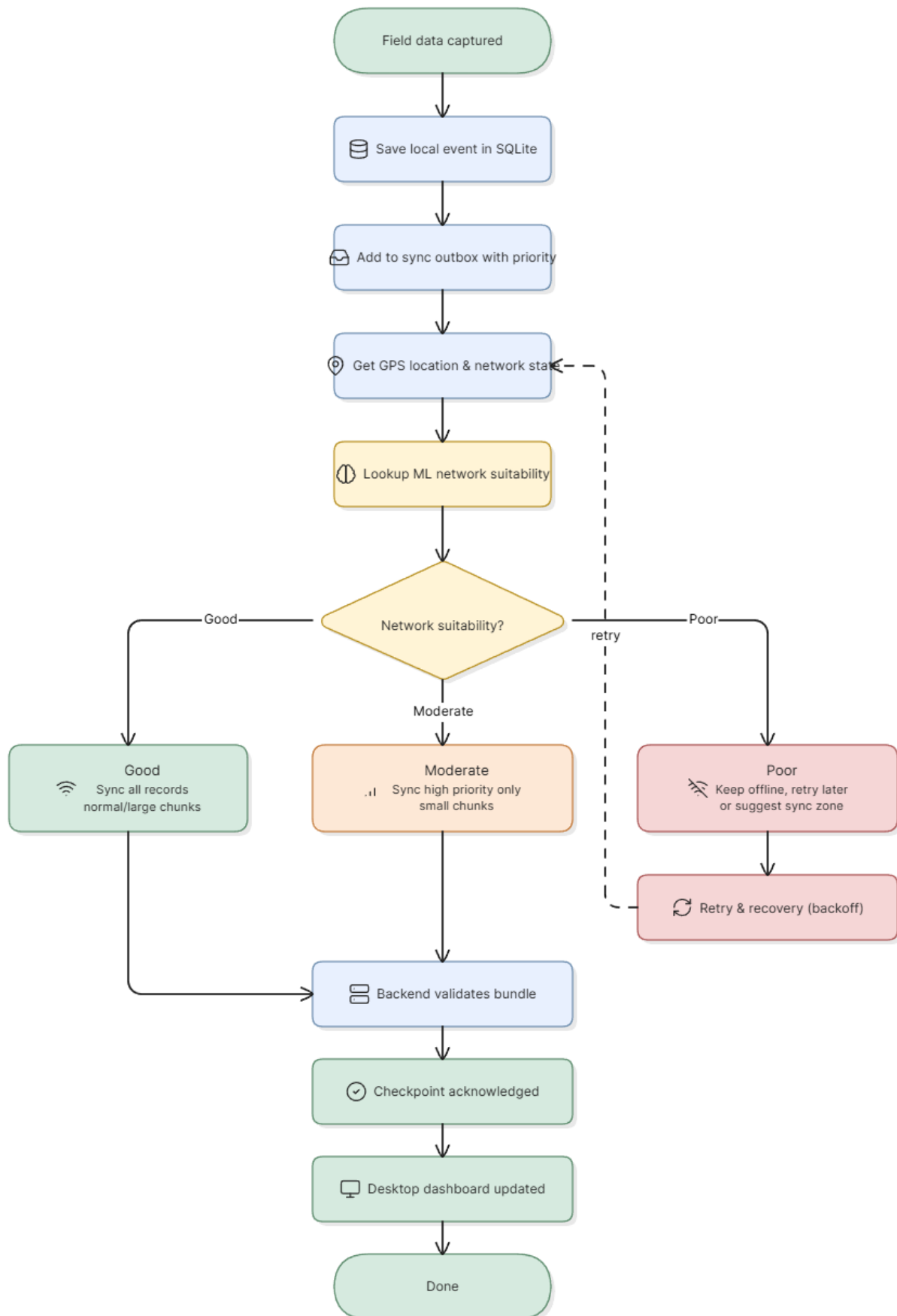


Figure 4-3: Adaptive synchronization decision flow using predicted suitability

The core runtime flow is as follows. First, the field collector enters collection or quality data in the mobile app, and the record is stored as a local event in SQLite. Second, the event is placed into the sync outbox with a priority label. Third, before synchronization, the app obtains the current GPS location and network state, maps the location to the nearest suitability tile, and obtains a Good, Moderate, or Poor suitability prediction. Fourth, the sync scheduler chooses a strategy: normal full sync, priority-only sync, reduced chunk-size sync, or delayed sync. Fifth, the cloud backend validates accepted bundles and updates projections. Finally, the desktop dashboard and analytics modules receive updated server-approved data.

5. USER REQUIREMENTS

Stakeholder identification

The primary stakeholders are field collectors and tappers using the offline-capable Flutter mobile app, collection-center and factory staff using connected dashboards, production managers relying on timely collection visibility, system administrators maintaining backend services, and the other research components that consume synchronized server-side data. The key stakeholder need is not only offline capture, but also guidance on when and how pending records should be synchronized under weak mobile coverage.

Table 5-1: Functional requirements of the component

ID	Requirement
FR1	The Flutter mobile client shall allow field users to create and review kithul collection records while fully offline.
FR2	The mobile client shall maintain a local SQLite event store, projections, sync outbox, checkpoint table, and priority tags for pending records.
FR3	The system shall ingest and preprocess real mobile network performance datasets and supporting cell-tower datasets for selected Sri Lankan field regions.
FR4	The ML module shall predict network suitability or future upload/latency performance for a given location or tile.
FR5	The mobile app shall display a sync suitability status such as Good, Moderate, or Poor together with a recommended sync action.
FR6	The mobile sync scheduler shall choose between sync all, high-priority-only sync, reduced chunk-size sync, and delayed sync based on predicted suitability and outbox context.
FR7	The app shall display a map layer showing weak and reliable synchronization zones for field collectors.
FR8	The cloud backend shall validate idempotent JSON delta bundles, suppress duplicates, persist accepted events, and return acknowledgements and checkpoints.
FR9	The system shall preserve correction-events and audit history without destructive overwrite.
FR10	The synchronized server-side projections shall be available to connected desktop, QR-can, spoilage prediction, and demand forecasting components.

Table 5-2: Non-functional requirements and target values

ID	Quality Attribute / Target
NFR1	Offline availability: core mobile field capture and local review functions shall remain available without network connectivity.
NFR2	Prediction responsiveness: the mobile app shall display suitability status within two seconds after location lookup or local model response.
NFR3	Model performance: the suitability classifier shall target at least 0.80 F1-score after threshold calibration using held-out real network data.
NFR4	Payload efficiency: the proposed sync strategy shall reduce unnecessary transfer attempts and avoid large

ID	Quality Attribute / Target
	bundles in predicted poor zones.
NFR5	Recoverability: interrupted mobile uploads shall resume from the latest acknowledged checkpoint rather than restarting from the beginning.
NFR6	Duplicate safety: retried bundles shall not create duplicate accepted events in the cloud event log.
NFR7	Security: mobile synchronization and desktop requests shall use authenticated HTTPS and role-restricted access to operational data.
NFR8	Privacy: exact farmer and collector locations shall be minimized or generalized where possible when used for network suitability analysis.
NFR9	Maintainability: model inputs, thresholds, sync policies, and event schemas shall be versioned.
NFR10	Scalability: the pilot shall support multiple mobile devices without requiring full-dataset resynchronization.

6. COMMERCIALIZATION PLAN

The immediate commercial opportunity for the overall platform lies in small and medium agro-processing businesses, cooperative collection networks, and rural food-processing companies that require source visibility but cannot rely on stable mobile connectivity at every farmer location. For this component, the commercial value is a smart synchronization layer that makes offline-first data capture more dependable and operationally useful.

The primary customer persona is a processing-site operations manager who oversees multiple field collectors and needs timely visibility of collection, quality, and intake records. The customer does not need to understand machine learning internals; the value proposition is practical: field records remain safe offline, critical records are uploaded sooner when possible, repeated failed uploads are reduced, and managers can see more reliable server-side data for production and payment decisions.

The component can be packaged as part of the full kithul traceability platform. A starter deployment could include the mobile offline-first app, basic cloud synchronization, and precomputed network suitability maps for selected collection regions. A growth deployment could include regularly updated suitability layers, additional analytics, support for multiple collection regions, and integration with production planning dashboards.

The competitive advantage is not merely offline storage. The advantage is combining offline-first field capture, real-data-driven network suitability prediction, priority-aware sync scheduling, and audit-friendly event convergence. This approach can later be adapted to other rural value chains such as spices, dairy, fisheries, or perishable produce where field connectivity affects operational data quality.

7. BUDGET AND JUSTIFICATION

Table 7-1: Estimated budget and justification

Item	Quantity / Duration	Estimated Cost (LKR)	Justification
AWS cloud integration environment (API Gateway, Lambda, PostgreSQL/PostGIS, logs)	Project period	20,000	Covers a minimal pilot deployment for validation, event storage, model API testing, and projection services.
Mobile data packages for real-device testing	Project period	3,000	Required to test sync decisions on real mobile networks in addition to public network datasets.

Item	Quantity / Duration	Estimated Cost (LKR)	Justification
Map and geospatial processing tools	Project period	0	Open-source tools such as QGIS, GeoPandas, PostGIS, and Leaflet/Flutter map libraries can be used.
Public network datasets	Project period	0	Ookla Open Data and OpenCelliD can be used subject to their license and attribution requirements.
Optional OpenCelliD API access / registration	Project period	0	OpenCelliD requires registration/API token for downloads, but no direct cash cost is expected for academic use.

The estimated direct cash requirement is approximately LKR 23,000. The cost remains feasible because the machine learning dataset is based on real public network performance data rather than paid proprietary measurements, and because development uses open-source tools and a minimal cloud pilot environment.

8. WORK BREAKDOWN STRUCTURE (WBS)

Table 8-1: Work packages and milestones for the component

WBS ID	Work Package	Key Output / Milestone
1.0	Problem analysis and literature consolidation	Validated problem statement, revised research gap, and IEEE-style reference base.
2.0	Real network dataset acquisition	Ookla mobile tile dataset and OpenCelliD data filtered to Sri Lanka / pilot regions.
3.0	Geospatial preprocessing and feature engineering	Cleaned tile-level dataset with temporal, spatial, reliability, and tower-density features.
4.0	Network suitability model development	Baseline and ML models for upload/latency regression or Good/Moderate/Poor classification.
5.0	Mobile offline data model and priority outbox	SQLite event store, outbox, checkpoints, and record-priority schema.
6.0	Adaptive sync scheduler implementation	Prediction-aware policy for sync all, priority-only sync, reduced chunk size, or delayed sync.
7.0	Cloud validation and projection pipeline	API Gateway/Lambda/PostgreSQL validation, idempotency, checkpoint, and projection services.
8.0	Mobile map and sync assistant UI	Network suitability map, pending records, recommended action, and nearby reliable sync zone display.
9.0	Evaluation and reporting	ML metrics, system performance metrics, diagrams, documentation, and dissertation-ready report.

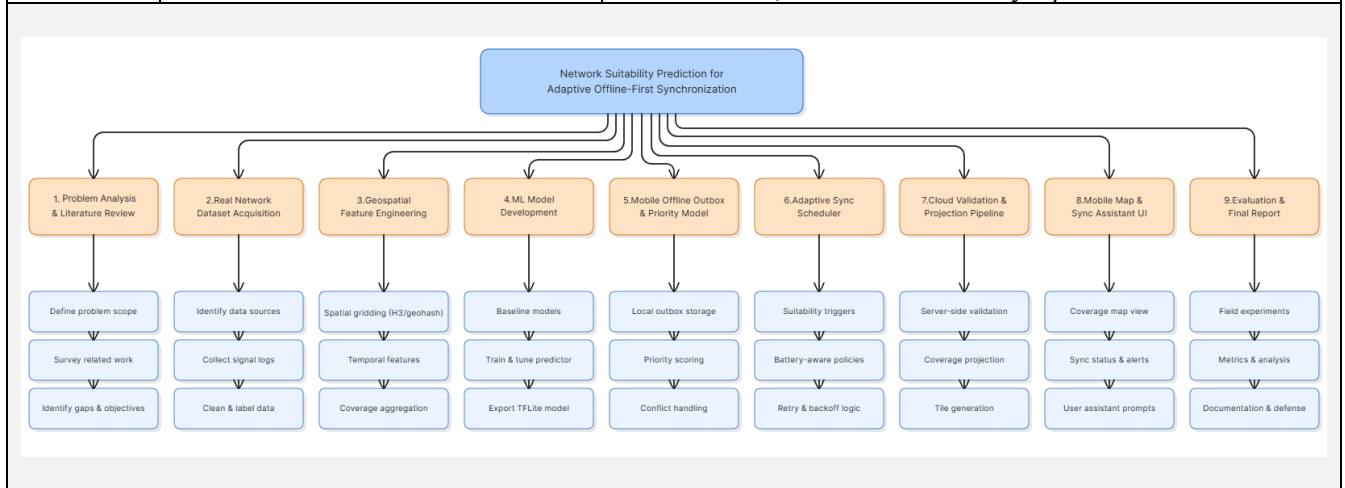


Figure 8-1: Work Breakdown Structure for the component

9. GANTT CHART

Table 9-1: Proposed semester Gantt chart for the component

Task	M1	M2	M3	M4	M5	M6
Literature review and requirement finalization	X	X				
Dataset acquisition and filtering	X	X				
Geospatial feature engineering		X	X			
Network suitability model training and evaluation			X	X		
Mobile offline outbox and priority sync design		X	X			
Adaptive sync scheduler implementation			X	X		
Cloud validation and projection integration				X	X	
Map UI and sync assistant integration				X	X	
Testing, evaluation, and reporting					X	X

- Milestone M1: revised literature review and component scope approved.
- Milestone M2: real network datasets acquired and preprocessing plan finalized.
- Milestone M3: feature-engineered dataset and baseline model completed.
- Milestone M4: suitability model and adaptive scheduler prototype implemented.
- Milestone M5: cloud integration, map UI, and sync assistant tested.
- Milestone M6: evaluation completed and final proposal/report artifacts finalized.

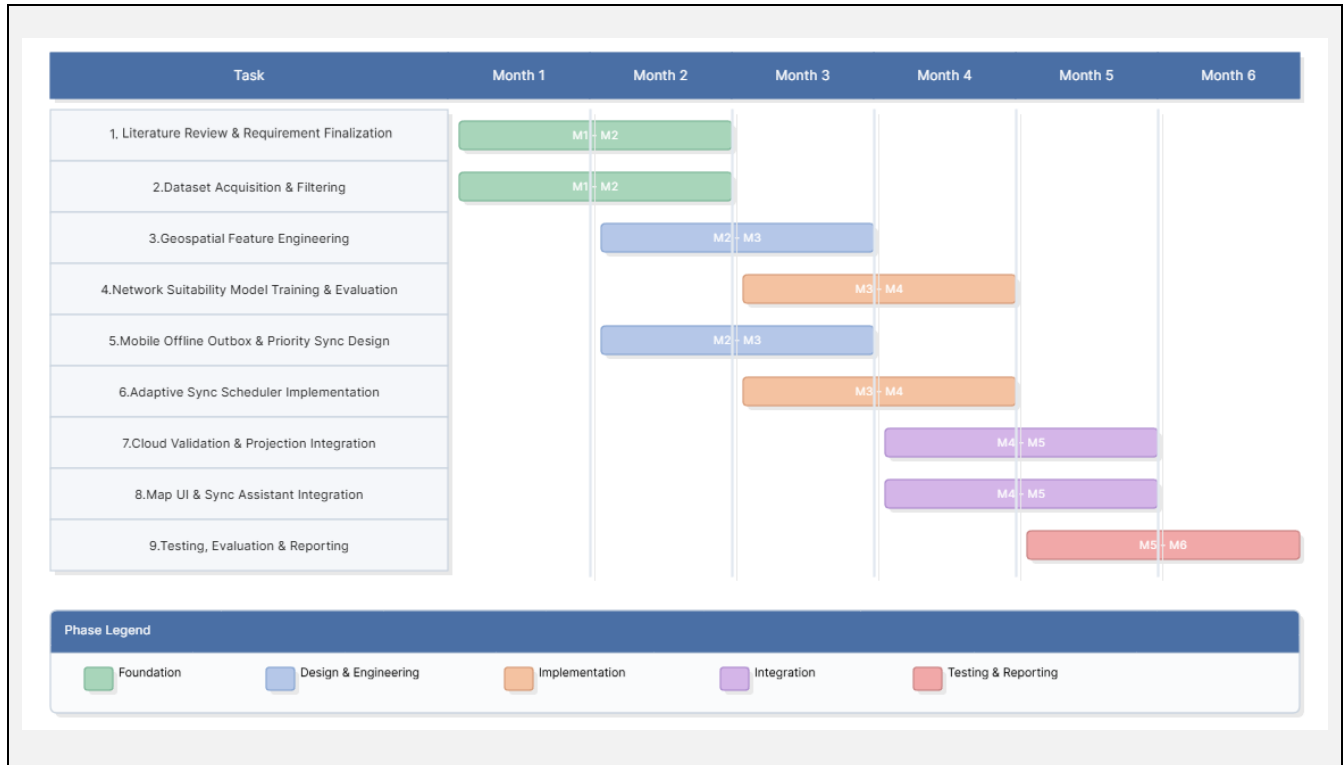


Figure 9-1: Semester Gantt chart for the component

10. REFERENCES LIST

- [1] ISO, ISO 22005:2007, Traceability in the feed and food chain - General principles and basic requirements for system design and implementation. Geneva, Switzerland: International Organization for Standardization, 2007.

- [2] GS1 AISBL, EPCIS and CBV Standard, Release 2.0.1. Brussels, Belgium: GS1, Jun. 2022.
- [3] R. Badia-Melis, P. Mishra, and L. Ruiz-Garcia, "Food traceability: New trends and recent advances. A review," *Food Control*, vol. 57, pp. 393-401, 2015, doi: 10.1016/j.foodcont.2015.05.005.
- [4] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proc. 2019 ACM SIGPLAN Int. Symp. New Ideas, New Paradigms, and Reflections on Programming and Software (Onward! 2019)*, Athens, Greece, 2019, pp. 154-178, doi: 10.1145/3359591.3359737.
- [5] N. Fraser, "Differential synchronization," in *Proc. 2009 ACM Symp. Document Engineering (DocEng 2009)*, Munich, Germany, 2009, pp. 13-20, doi: 10.1145/1600193.1600198.
- [6] M. Shapiro, N. Preguica, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in *Stabilization, Safety, and Security of Distributed Systems, LNCS 6976*. Berlin, Germany: Springer, 2011, pp. 386-400, doi: 10.1007/978-3-642-24550-3_29.
- [7] P. S. Almeida, A. Shoker, and C. Baquero, "Delta state replicated data types," *J. Parallel Distrib. Comput.*, vol. 111, pp. 162-173, 2018, doi: 10.1016/j.jpdc.2017.08.003.
- [8] M. Kleppmann and A. R. Beresford, "A conflict-free replicated JSON datatype," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 10, pp. 2733-2746, Oct. 2017, doi: 10.1109/TPDS.2017.2697382.
- [9] D. B. Terry, M. Theimer, K. Petersen, A. J. Demers, M. J. Spreitzer, and C. H. Hauser, "Managing update conflicts in Bayou, a weakly connected replicated storage system," in *Proc. 15th ACM Symp. Operating Systems Principles (SOSP 1995)*, Copper Mountain, CO, USA, 1995, pp. 172-183, doi: 10.1145/224056.224070.
- [10] Y. Saito and M. Shapiro, "Optimistic replication," *ACM Comput. Surv.*, vol. 37, no. 1, pp. 42-81, Mar. 2005, doi: 10.1145/1057977.1057980.
- [11] Ookla, "Speedtest by Ookla Global Fixed and Mobile Network Performance Map Tiles," GitHub, 2026. [Online]. Available: <https://github.com/teamookla/ookla-open-data>. Accessed: May 10, 2026.
- [12] AWS Open Data Registry, "Speedtest by Ookla Global Fixed and Mobile Network Performance Maps," 2026. [Online]. Available: <https://registry.opendata.aws/speedtest-global-performance/>. Accessed: May 10, 2026.
- [13] OpenCellID, "Data Downloads - OpenCellID," 2026. [Online]. Available: <https://opencellid.org/downloads>. Accessed: May 10, 2026.
- [14] Android Developers, "Read network state," Android Connectivity Documentation, 2026. [Online]. Available: <https://developer.android.com/develop/connectivity/network-ops/reading-network-state>. Accessed: May 10, 2026.
- [15] International Telecommunication Union, *Measuring digital development: Facts and Figures 2024*. Geneva, Switzerland: ITU, 2024.
- [16] Department of Census and Statistics, Sri Lanka, *Computer Literacy Statistics 2024*. Battaramulla, Sri Lanka, 2025.
- [17] E. Eyceyurt, Y. Egi, and J. Zec, "Machine-learning-based uplink throughput prediction from physical layer measurements," *Electronics*, vol. 11, no. 8, Art. no. 1227, Apr. 2022, doi: 10.3390/electronics11081227.
- [18] D. Minovski, N. Ogren, C. Ahlund, and K. Mitra, "Throughput prediction using machine learning in LTE and 5G networks," *IEEE Trans. Mobile Comput.*, 2021, doi: 10.1109/TMC.2021.3099397.
- [19] Google Firebase, "Access data offline," Cloud Firestore Documentation, 2026. [Online]. Available: <https://firebase.google.com/docs/firestore/manage-data/enable-offline>. Accessed: May 10, 2026.
- [20] Apache Software Foundation, "Replication," Apache CouchDB 3.5 Documentation, 2026. [Online]. Available: <https://docs.couchdb.org/en/stable/replication/index.html>. Accessed: May 10, 2026.
- [21] Amazon Web Services, "Conflict detection and resolution in AWS AppSync," AWS AppSync Developer Guide, 2026. [Online]. Available: <https://docs.aws.amazon.com/appsync/latest/devguide/conflict-detection-and-resolution.html>. Accessed: May 10, 2026.

11. APPENDICES

Appendix A: Proposed Network Suitability Dataset Schema

Table 11-1: Proposed network suitability dataset schema

Field	Example / Meaning	Purpose
quadkey	Unique tile identifier from Ookla map tiles	Input key
tile_x, tile_y	Tile centroid coordinates	Spatial feature
avg_u_kbps	Average upload speed in kilobits per second	Regression target / feature
avg_d_kbps	Average download speed in kilobits per second	Feature
avg_lat_ms	Average latency in milliseconds	Regression target / feature
tests	Number of tests contributing to tile aggregate	Reliability feature
devices	Number of unique devices contributing tests	Reliability feature
year, quarter	Time period of performance aggregate	Temporal feature
prev_avg_u_kbps	Previous-quarter upload speed for same tile	Lag feature
upload_rolling_avg_3q	Rolling average upload speed for three quarters	Trend feature
nearest_tower_km	Distance to nearest OpenCellID tower	Infrastructure feature
tower_count_3km	Number of towers within 3 km of tile centroid	Infrastructure feature
suitability_class	Good, Moderate, or Poor based on calibrated operational thresholds	Classification target

Appendix B: Example Adaptive Synchronization Decision Manifest

```
{
  "device_id": "mob-col-07",
  "current_quadkey": "1230321012302101",
  "predicted_suitability": "Moderate",
  "predicted_upload_kbps": 920,
  "predicted_latency_ms": 185,
  "pending_records": 18,
  "high_priority_records": 6,
  "selected_strategy": "PRIORITY_ONLY_SYNC",
  "bundle_policy": {
    "chunk_size_kb": 64,
    "max_in_flight_chunks": 1,
    "retry_backoff": "conservative"
  },
  "reason": "Moderate predicted upload suitability; critical records synchronized first."
}
```

This manifest illustrates how the mobile client can combine predicted suitability and pending outbox context before selecting a synchronization strategy.

Appendix C: Proposed Mobile-Local and Cloud Schema Summary

Table 11-2: Proposed mobile-local and cloud schema summary

Table Name	Location	Key Fields	Purpose
event_log	Mobile / Cloud	event_id, aggregate_type, aggregate_id, event_type, hlc, payload hash	Stores immutable operational and correction-events as the primary audit trail.
sync_outbox	Mobile	event_id, priority, bundle_state, retry_count, last_attempt_at	Tracks pending local events and their synchronization priority.
sync_checkpoint	Mobile / Cloud	device_id, last_server_checkpoint, last_downsync_at	Maintains durable synchronization progress for mobile replicas.
network_suitability_cache	Mobile	quadkey, suitability_class, predicted_upload_kbps,	Enables local suitability lookup and map display.

Table Name	Location	Key Fields	Purpose
		predicted_latency_ms, updated_at	
network_tile_features	Cloud	quadkey, year, quarter, avg_u_kbps, avg_d_kbps, avg_lat_ms, tests, devices	Stores cleaned public network performance data.
tower_feature_store	Cloud	quadkey, nearest_tower_km, tower_count_1km, tower_count_3km	Stores engineered OpenCellID infrastructure features.
bundle_receipt	Cloud	bundle_id, device_id, idempotency_key, accepted_at	Prevents duplicate effects when clients retry the same bundle.
conflict_review	Cloud	review_id, aggregate_id, event_ids, review_status	Retains semantically unsafe conflicts without deleting evidence.
projection_collection	Mobile / Cloud	collection_id, can_id, farmer_id, qty_effective, status	Materialized current-state view for collection-related UI queries.
projection_can	Mobile / Cloud	can_id, lifecycle_state, last_known_center, active_flag	Current can status and traceability-friendly lookup view.

In the proposed design, public network performance data and derived suitability features are stored separately from operational kithul business records. The mobile app may cache only the suitability tiles required for selected collection routes, while the cloud backend retains the full model outputs and source feature tables for retraining and auditability.